

DFTL: A Flash Translation Layer Employing Demand-based Selective Caching of Page-level Address Mappings

Aayush Gupta, Youngjae Kim, Bhuvan Urgaonkar

2026. 07. 28

Presentation by Jung In, Minsu Kim

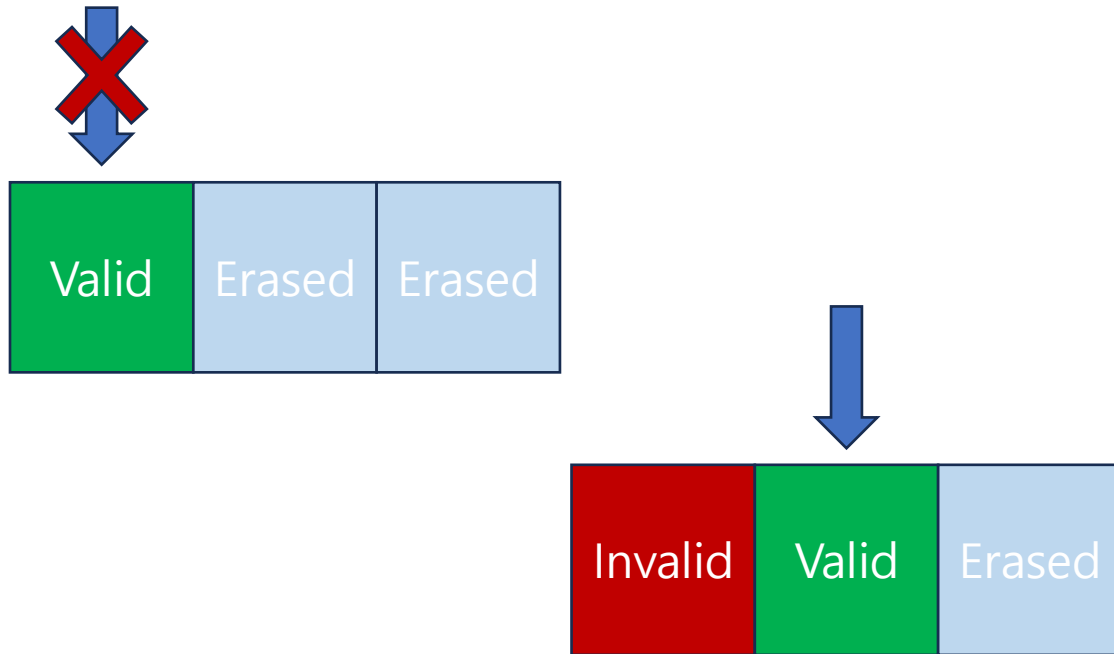
wjddls0318@dankook.ac.kr, minsk0515@dankook.ac.kr

Contents

1. Problem Background
2. Core Idea of DFTL
3. DFTL Architecture
4. DFTL Principles: Address Translation Process
5. Experiment Results
6. Conclusion

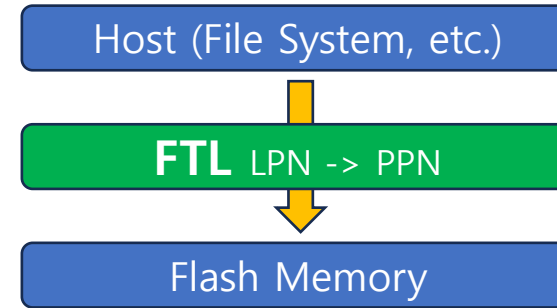
Problem Background

Nand Flash의 Out-of-place Update



Flash 장치의 Write는 Erased 상태의 Page에서만 수행 가능

FTL

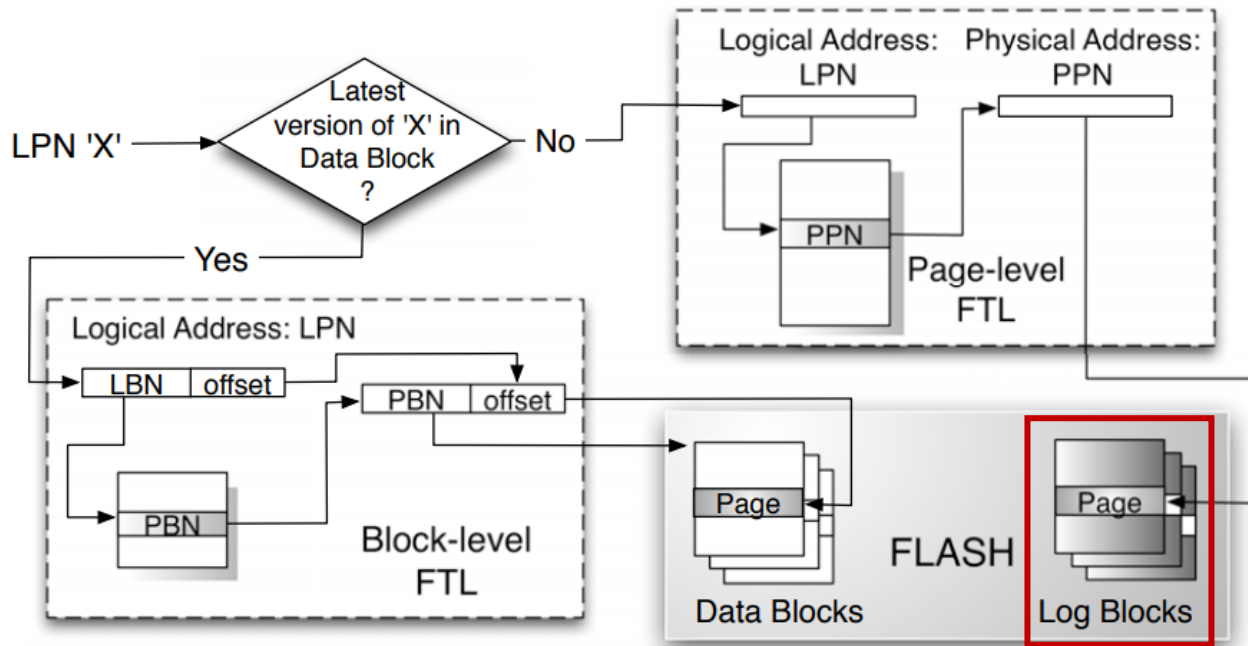


- Host의 논리 주소를 Flash의 물리 주소로 변환
- Out-of-place update 관리
- Garbage Collection 수행

Mapping Type	특징
Page 수준 Mapping	SRAM 필요 용량 증가
Block 수준 Mapping	내부 단편화, GC Overhead 증가
Hybrid Mapping	Data Block(Block) + Update Block(Page)

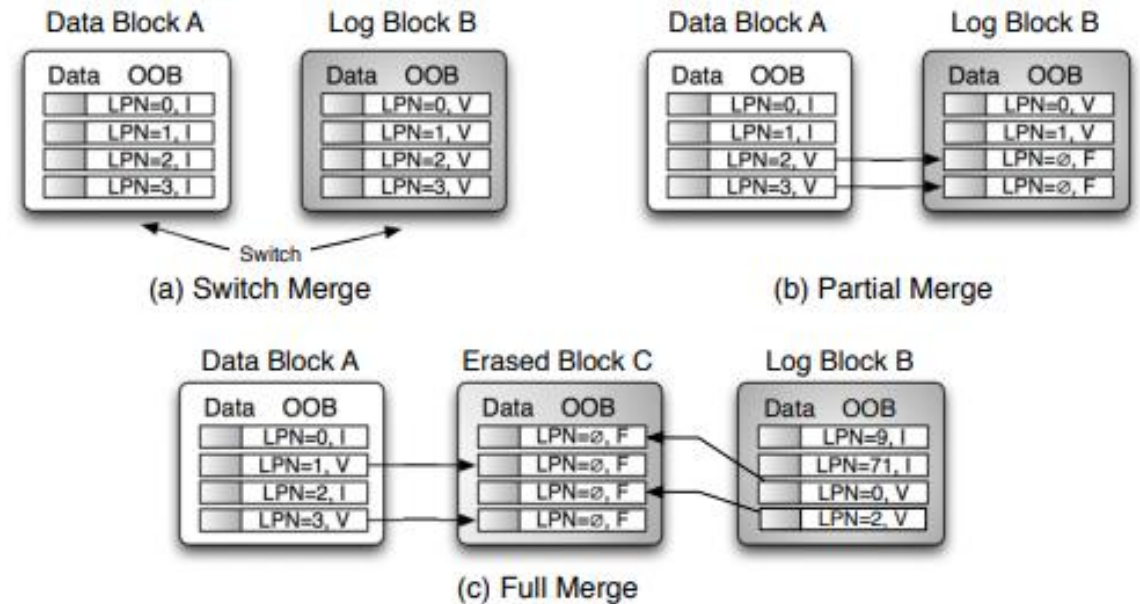
Problem Background

Hybrid FTL의 기본 구조



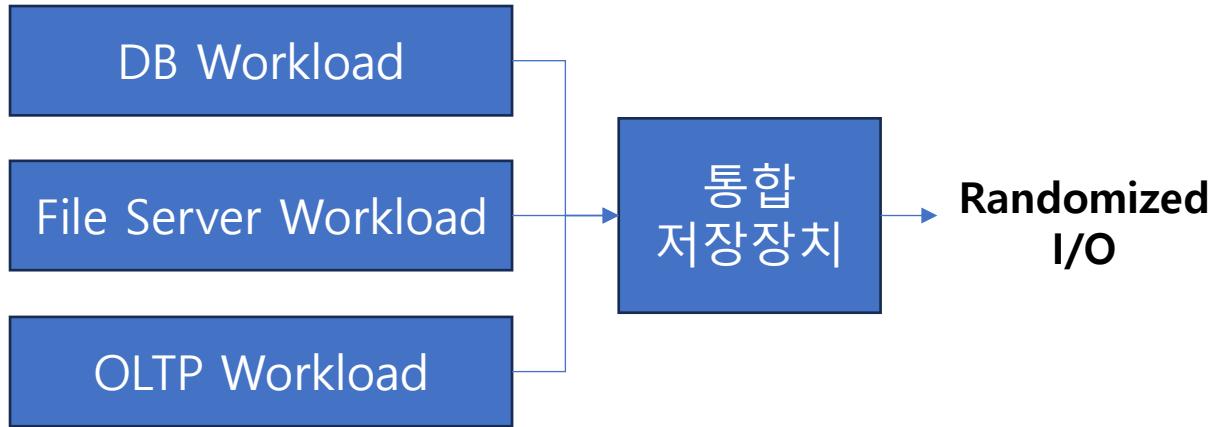
공간 부족 시, GC 수행

Merge 종류



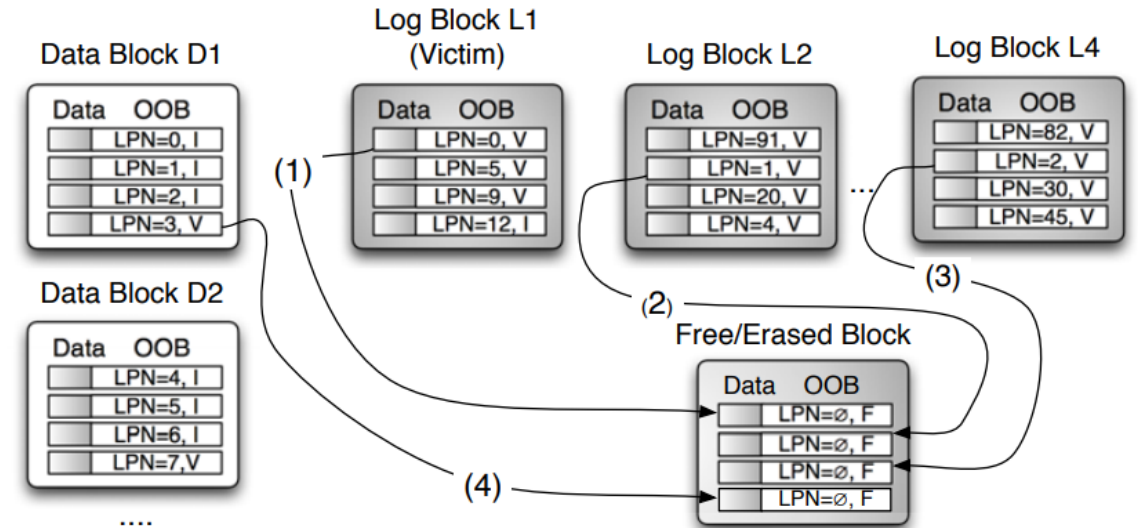
Problem Background

Enterprise-Scale Workload



Write 비중이 높은 워크로드
Temporal Locality

Hybrid FTL의 한계 (FAST)



Hybrid 구조를 유지하여 Full Merge를 해결 시도 (BAST, FAST..)
→ GC Overhead, 낮은 시간적 지역성 활용

Core Idea of DFTL

"doing away with log-blocks."

DFTL Architecture

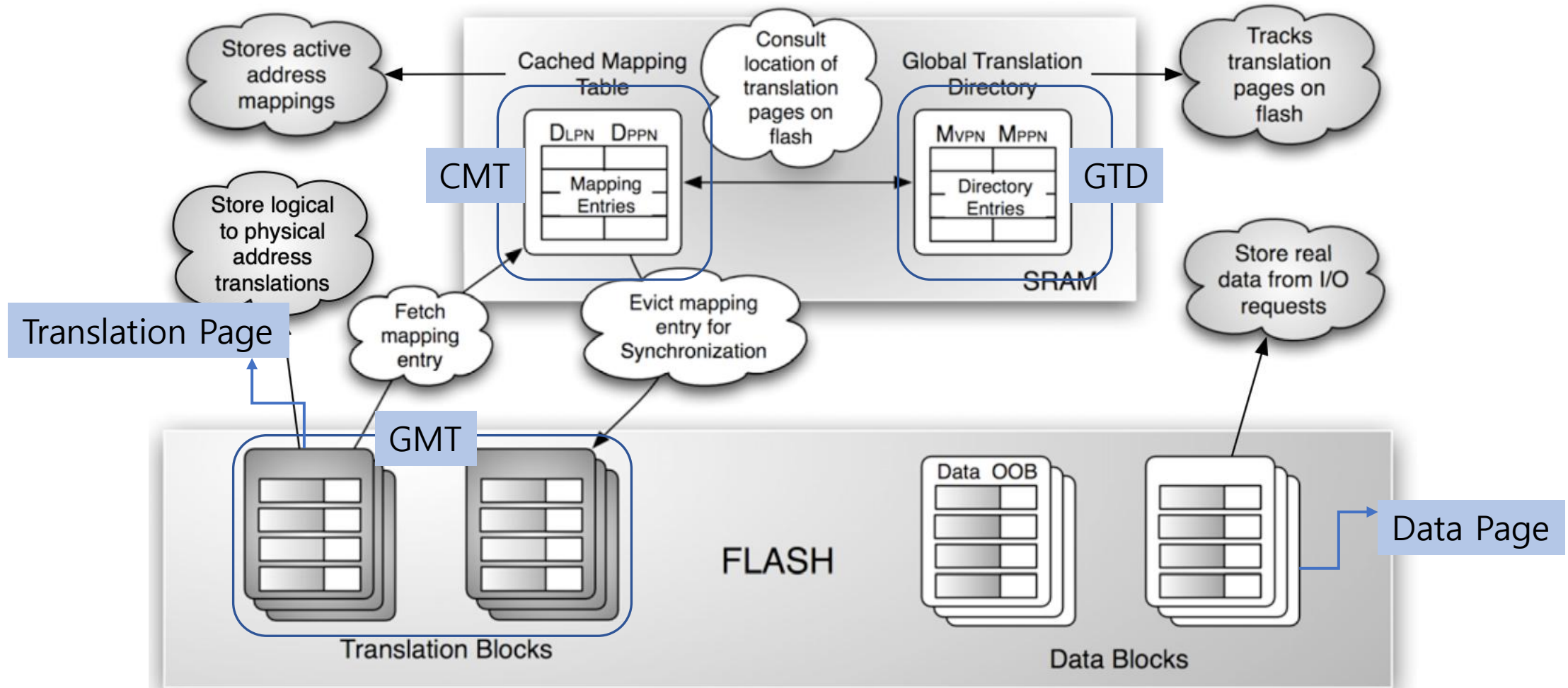
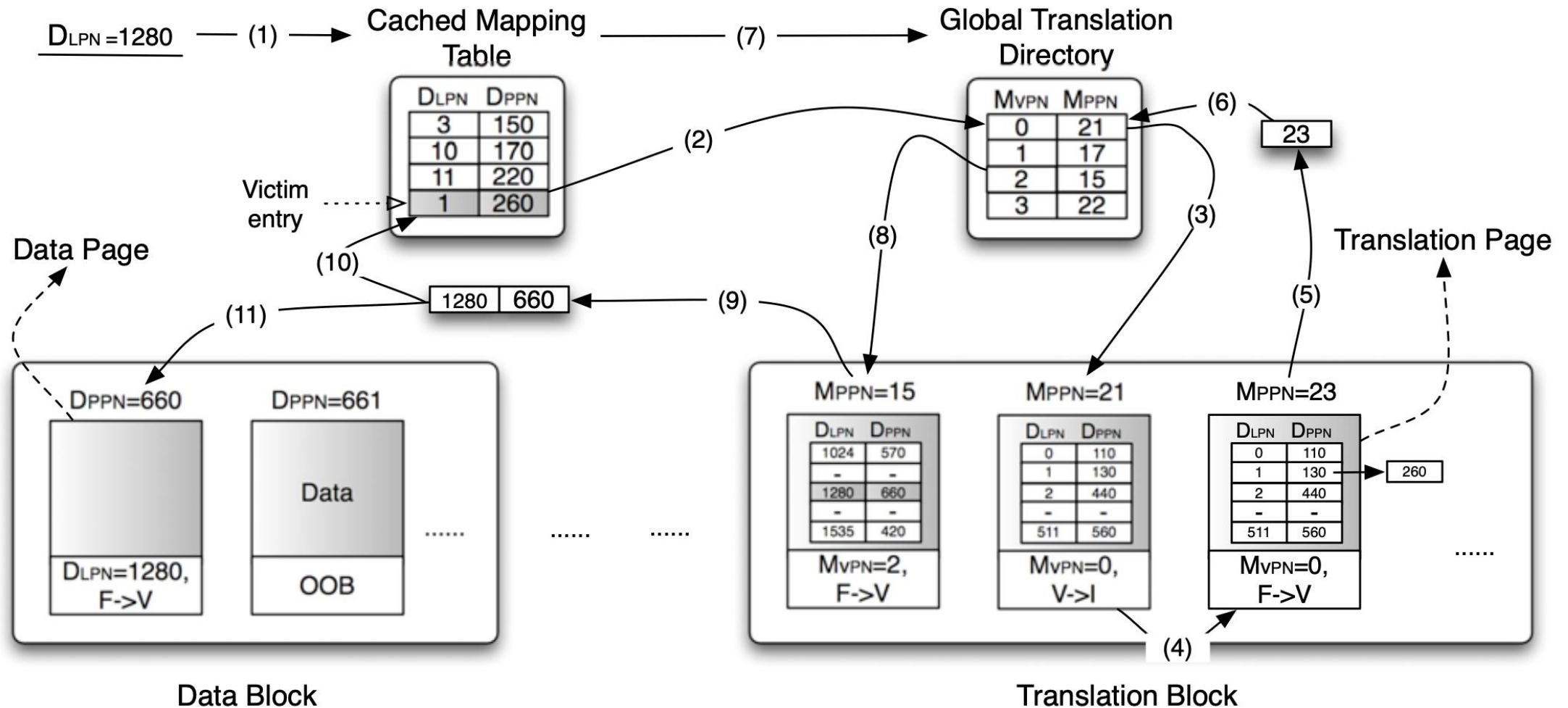


Figure 4: Schematic Design of DFTL

Address Translation Process



Address Translation Process – Overhead

Victim state = clean

1. Original request에 대한 mapping entry 적재를 위해 Translation Page 읽기

Overhead: Read * 1

Victim state = dirty

1. victim entry 업데이트를 위한 Translation Page 원본 읽기
2. 업데이트가 반영된 Translation Page 쓰기
3. Original request에 대한 mapping entry 적재를 위해 Translation Page 읽기

Overhead: Read * 2 + Write * 1

Read/Write Operation

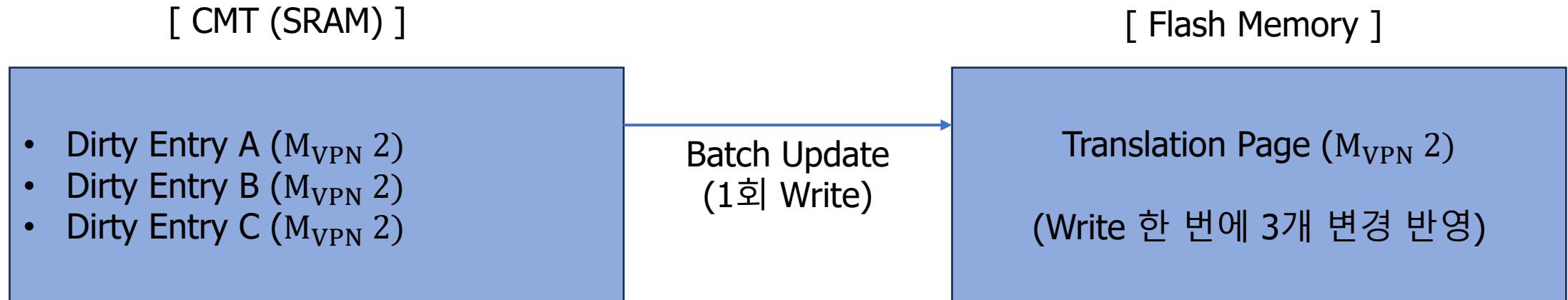
- Read의 경우, 주소 변환이 수행된 후 해당하는 Data Page를 읽음
- DFTL은 'Current Data Block'과 'Current Translation Block'두 가지 블록을 유지함
- 이는 각각 Data Page와 Translation Page가 쓰여지는 블록이며
- Write Request의 경우 Current Data Block 안의 새 Data Page에 Write 하고, CMT에 있는 mapping entry를 업데이트함
- 이 특성으로 인해 temporal locality 활용 용이해짐

Read/Write Operation – lazy copy

- Mapping entry를 수정해야 할 때 CMT에 해당 mapping entry가 존재하면 GMT상의 mapping entry는 수정되지 않음
- CMT상의 mapping entry가 evict될 때 수정 사항이 GMT에 반영
- 이를 lazy copy라고 함
- Lazy copy로 인해 GMT 업데이트는 오직 한 번 일어남

Read/Write Operation – batch update

- 'dirty' state의 mapping entry가 evict될 때, 같은 Translation Page에 속하는 entry들을 한 번에 업데이트하는 것
- 한 번의 Translation Page 업데이트로 여러 개의 mapping entry 변경 사항을 업데이트 할 수 있어 Write 횟수가 감소함



Garbage collection

- Write/Update가 수행되며 시간이 지남에 따라, 가용 가능한 물리 블록 수는 줄어들 것
- GC 오버헤드를 고려하여, 가용 Free Block 수가 임계값 ($GC_{threshold}$) 이하로 떨어졌을 때 GC 트리거
- Victim Block의 종류에 따라 처리가 다름
- Data Block: valid page 복사 후, 해당 Page들과 관련된 모든 Translation Page와 CMT 업데이트를 진행
- Translation Block: valid Page 복사 후, GTD 업데이트

Comparison of Hybrid FTLs with DFTL

Metrics	Hybrid FTL	DFTL
Full Merge	Full Merge 연산을 줄이려 시도하지만, 여전히 발생	완전 제거
Partial Merge & Hot/Cold (temporal locality)	외부 별도 메커니즘 필요(LAST, Superblock)	Temporal Locality 기반 hot/cold 암묵적 자동 분리
Random Write Performance	Log block의 비싼 Merge 연산으로 인해 성능 저하 발생	Full Merge 제거로 Random Write 성능 향상
Block Utilization	Log block만 업데이트 수용 가능하여 block utilization 제한됨	모든 블록이 업데이트 수용 가능

The FlashSim Simulator

- [Objective & Base Framework]
 - 다양한 FTL Scheme의 성능 파급효과(Performance Implication) 정밀 평가
 - 타 연구에서 널리 사용된 event-driven 시뮬레이터인 Disksim에 기반하여 자체 시뮬레이션 프레임워크를 구축
- [FlashSim Architecture & Simulation Scope]
 - 전체 플래시 저장 환경을 모듈화하여 묘사
 - device drivers, controllers, caches, flash devices, interconnects

Experiment Results

[Environments]

- **Flash Settings**
 - 32GB NAND Flash
 - 일부 공간만 워크로드 저장되는 활성 영역으로 설정
- **SRAM Settings**
 - FAST 주소변환 정보 저장 시 필요한 최소 크기로 설정
- **Test FTL**
 - FAST: 완전 연관형 Hybrid FTL
 - DFTL: 수요 기반 페이지 매핑 FTL
 - Baseline: 이상적인 FTL (순수 페이지 기반 FTL)

• Test Workloads

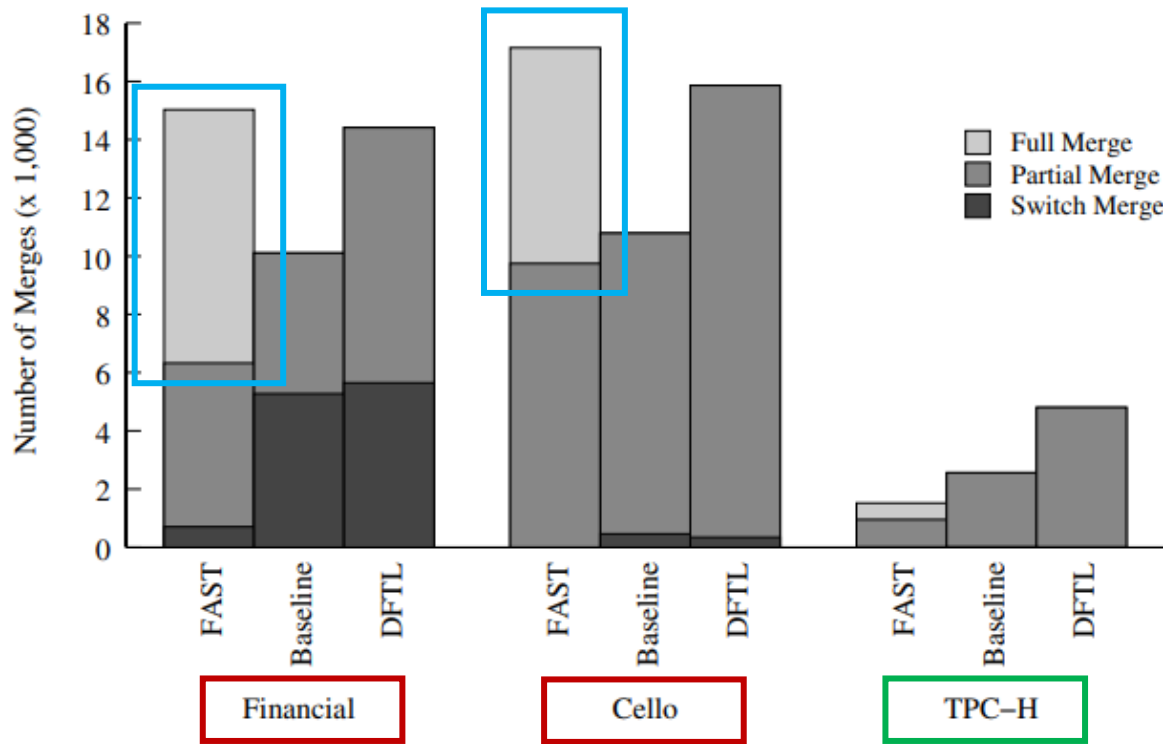
Workload	Definition	Features
Financial	금융 OLTP 쓰기 Trace	랜덤 쓰기가 매우 많은 OLTP
Cello99	시분할 서버 디스크 Trace	쓰기가 많고 요청 강도가 높음
TPC-H	OLAP 의사결정 Benchmark	읽기 중심 OLAP
Web Search	읽기 중심 웹 검색 Trace	거의 대부분 읽기 요청

• Main Performance Metrics

- ✓ **GC 효율 및 FTL 내부 오버헤드**
 - 병합 횟수와 유형, 블록 삭제 횟수
 - 유효 페이지 복사와 추가 Translation Read/Write
- ✓ **시스템 응답 시간**
 - 장치 서비스 시간 + I/O Driver 큐잉 지연시간

Experiment Results

[Experiment 1: FTL 간 GC 및 주소 변환 Overhead 분석]



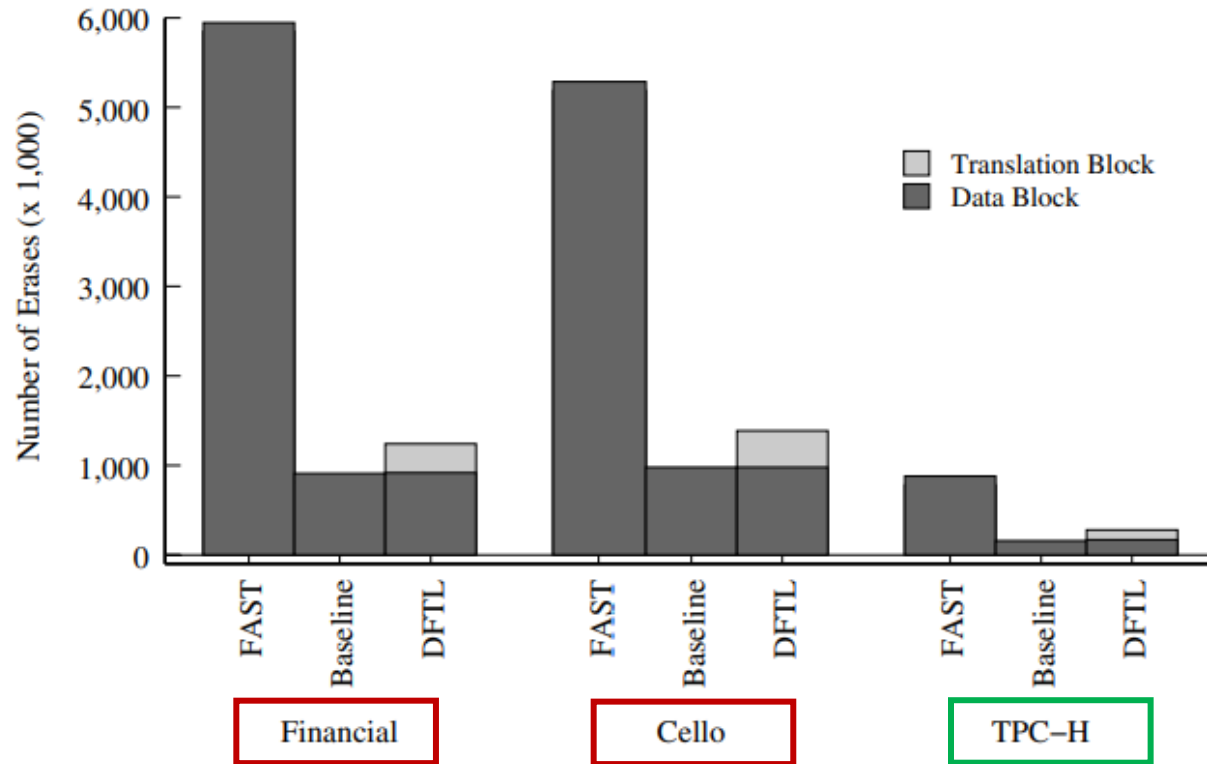
(a) Merge Operations

Merge 횟수는 FAST와 DFTL이 유사 or DFTL이 더 많음

DFTL에는 Full Merge가 발생하지 않음

Experiment Results

[Experiment 1: FTL 간 GC 및 주소 변환 Overhead 분석]



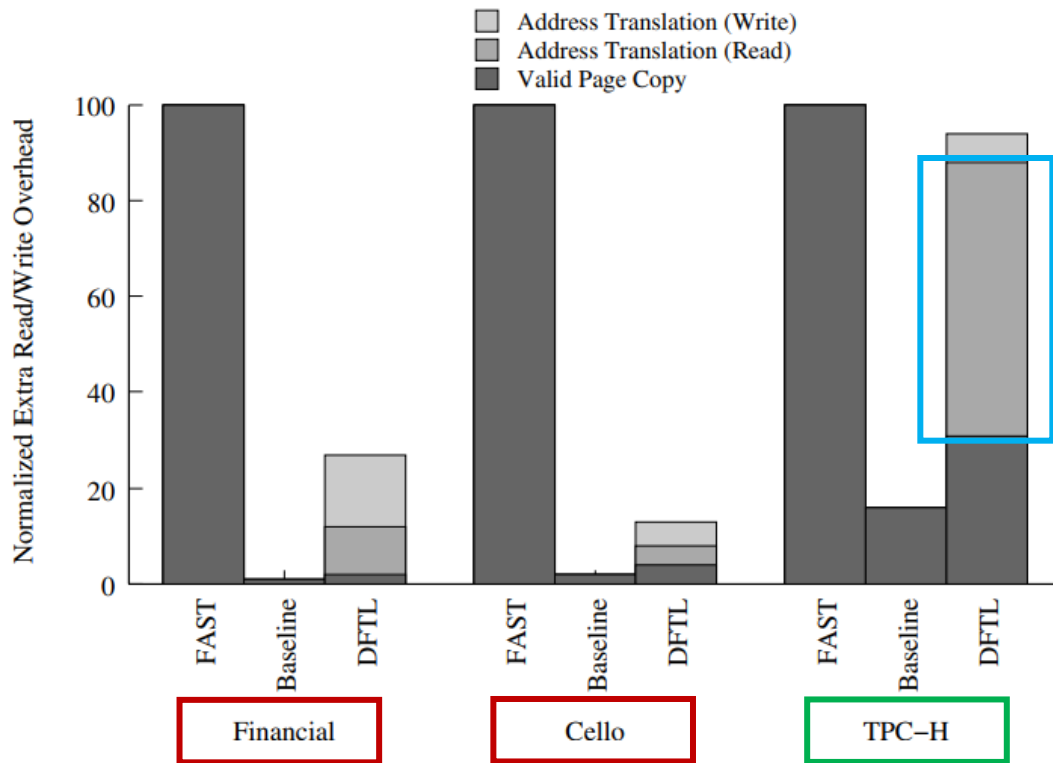
(b) Block Erases

FAST는 Full Merge로 인한 삭제 횟수 증가

DFTL은 FAST의 1/3 수준

Experiment Results

[Experiment 1: FTL 간 GC 및 주소 변환 Overhead 분석]



(c) Extra Read/Write Operations

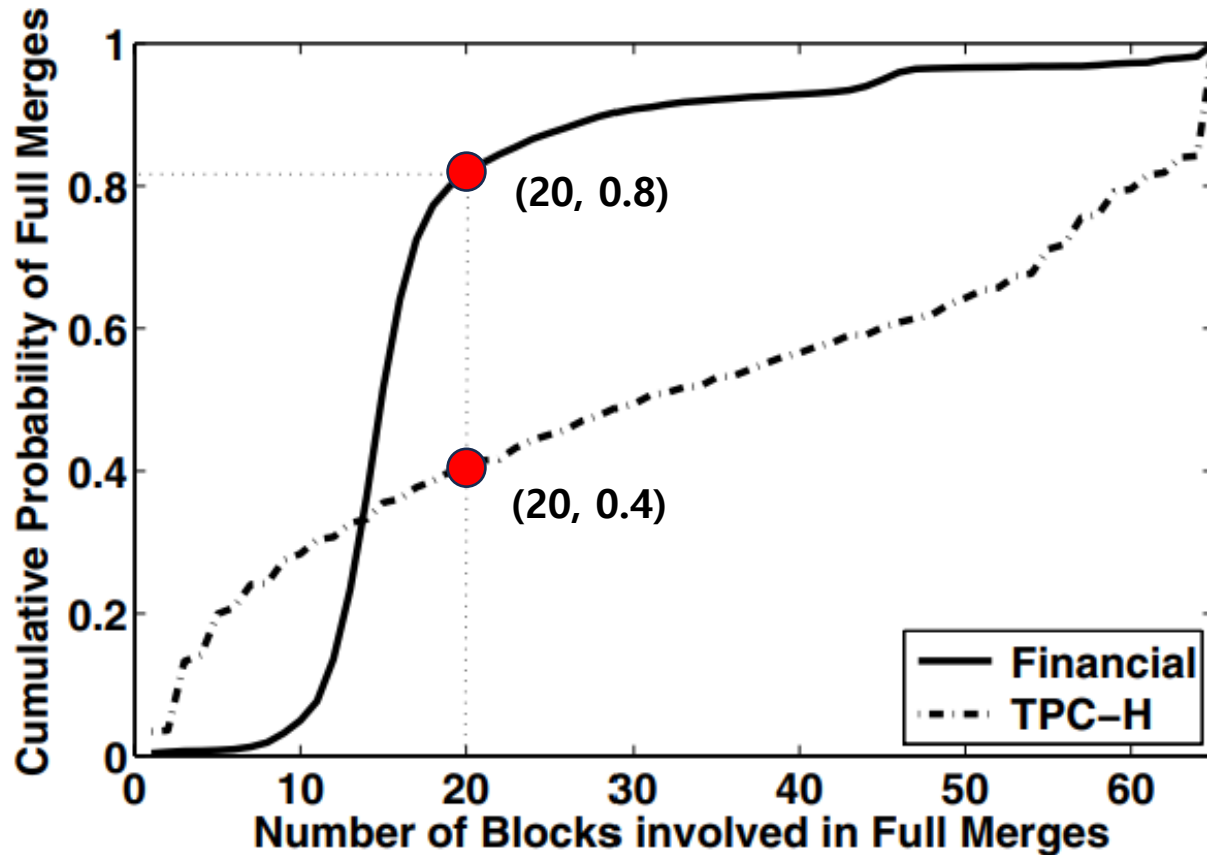
- Addr Translation Read: 매핑 정보 읽기
- Addr Translation Write: 매핑 정보 갱신
- Valid Page Copy: 유효 페이지 정보 복사

Write 중심 워크로드에서 DFTL은
FAST의 27%, 13% 수준의 Overhead

Read 중심 워크로드에서는
FAST와 DFTL의 추가 R/W Overhead 차이가 제한적

Experiment Results

[Experiment 1_Sub: Full Merge 한 번에 포함되는 데이터 블록 수]



- 막대그래프 전체: 총 Full Merge 발생 횟수
- 보라색: 각 Full Merge 마다 20개 초과 데이터 블록 포함
- 하늘색: 각 Full Merge 마다 20개 이하 데이터 블록 처리

- Financial (20, 0.8) 위치



- TPC-H (20, 0.4) 위치

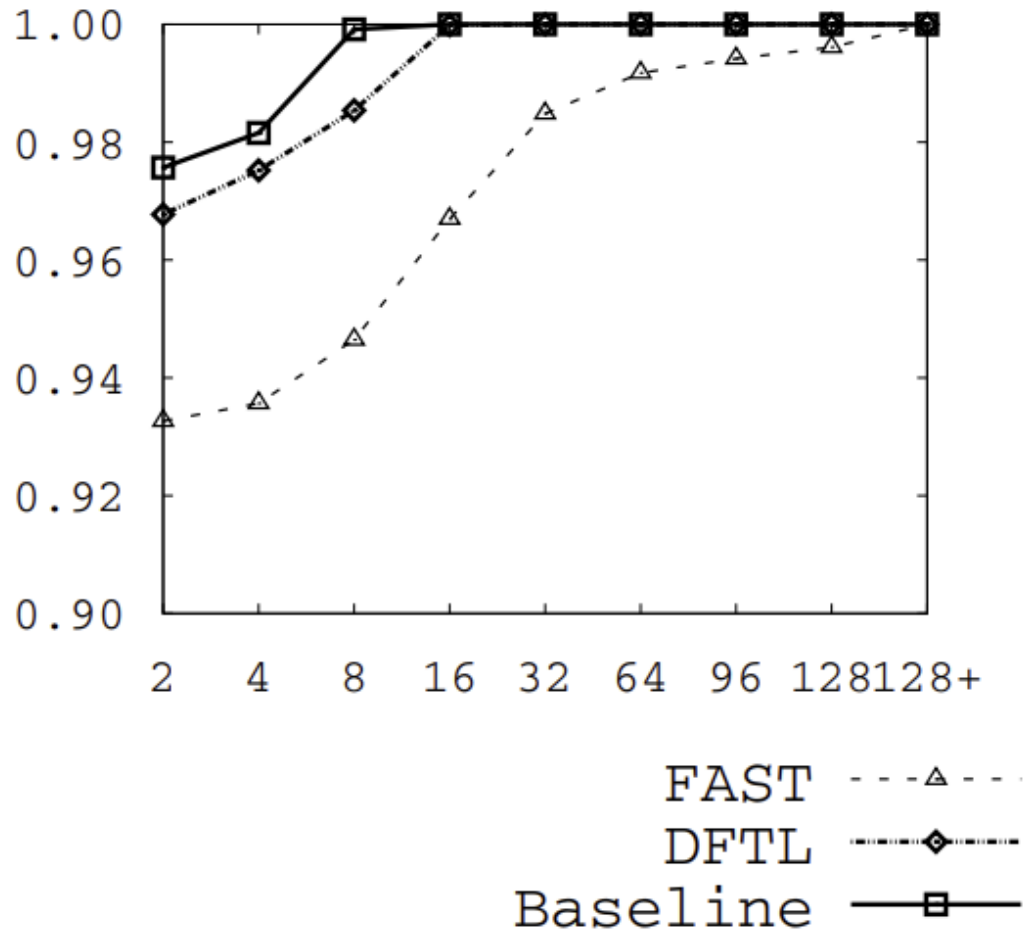


(Merge 100회 수행 가정. 20개 이하 = 10개 / 20개 초과 = 21개 일반화)

Merge Type	Calculation	Total Blocks
Full Merge (Financial)	$20 \times 21 + 80 \times 10$	1220개
Full Merge (TPC-H)	$60 \times 21 + 40 \times 10$	1660개
Switch/Partial	100×1	100개

Experiment Results

[Experiment 2: 성능 분석 (Financial Trace)]



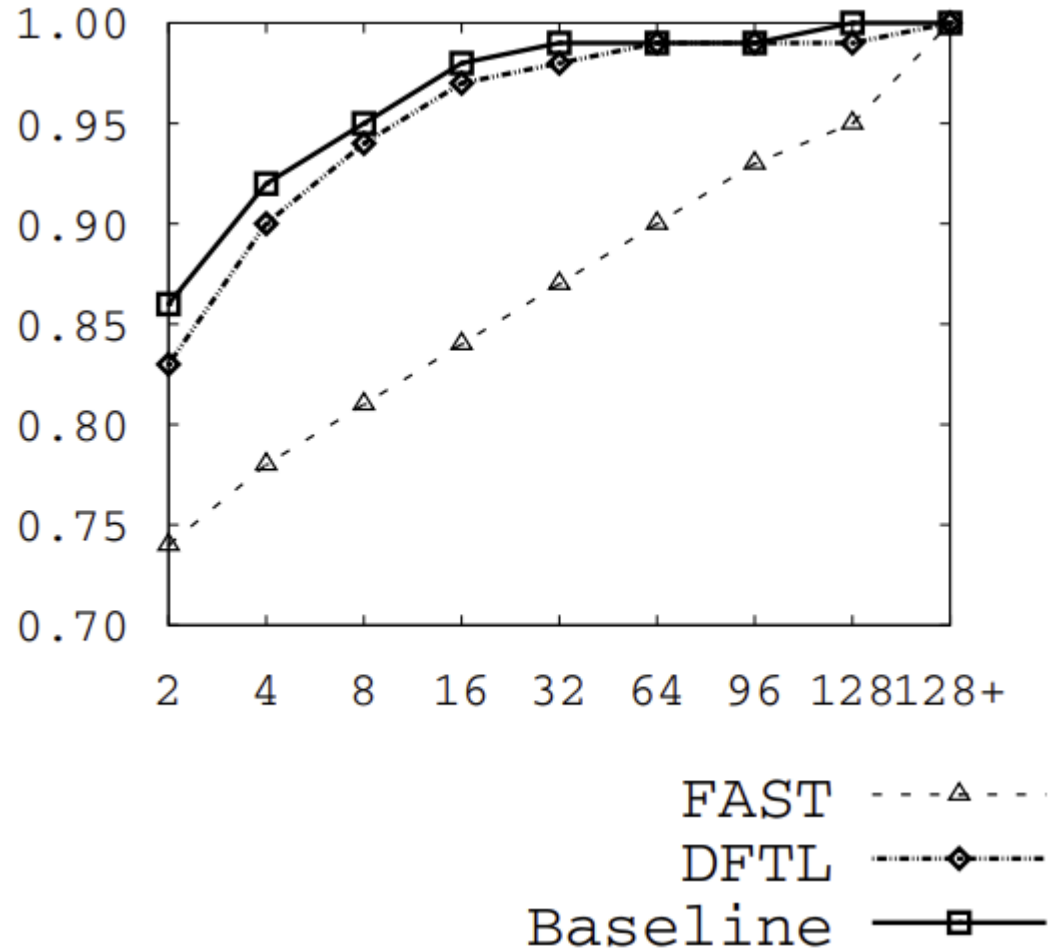
(Financial: Write 91% + 평균 요청 간격 133ms)

DFTL은 16ms 이후 모든 요청 처리
FAST는 128ms에서도 완료 X

→ DFTL이 FAST 비해 전체 I/O 응답 시간 78% 향상

Experiment Results

[Experiment 2: 성능 분석 (Cello99)]



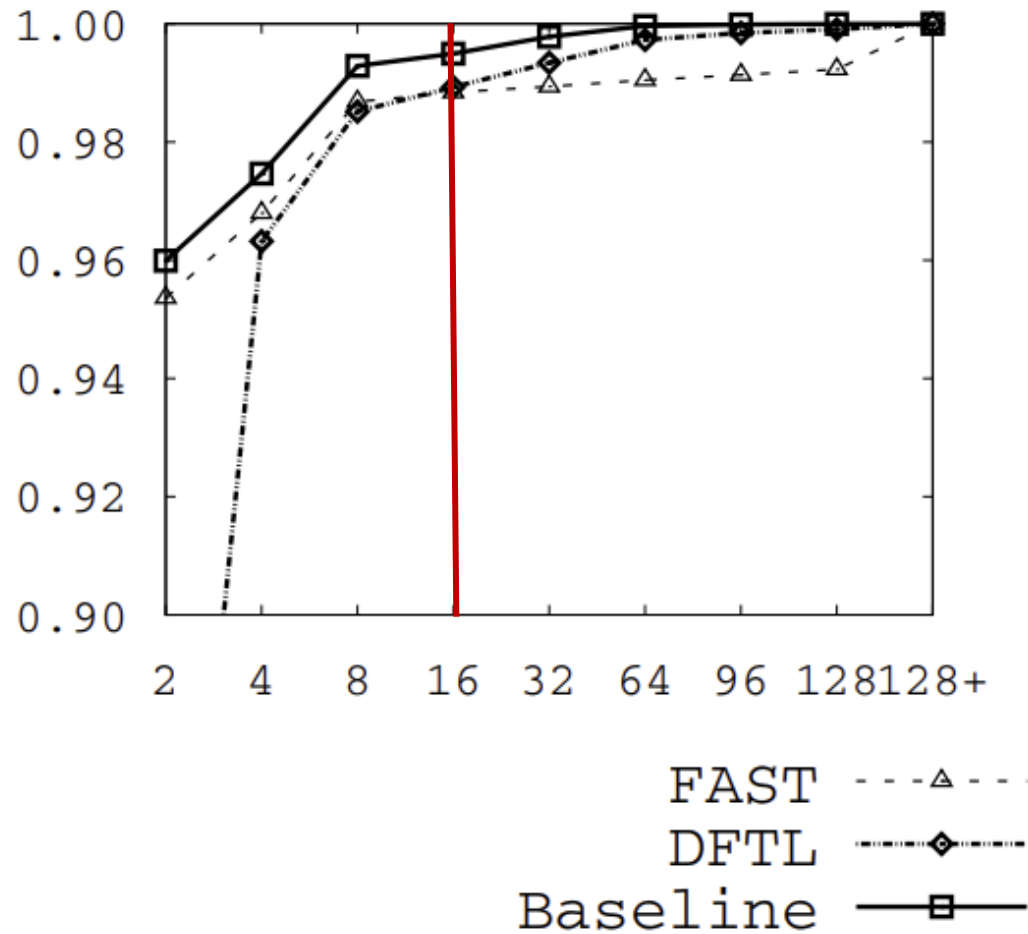
(Cello99: Write 65% + 평균 요청 간격 4ms)

더 높은 강도의 write 워크로드

→ DFTL과 FAST의 성능 차이가 커짐

Experiment Results

[Experiment 2: 성능 분석 (TPC-H)]



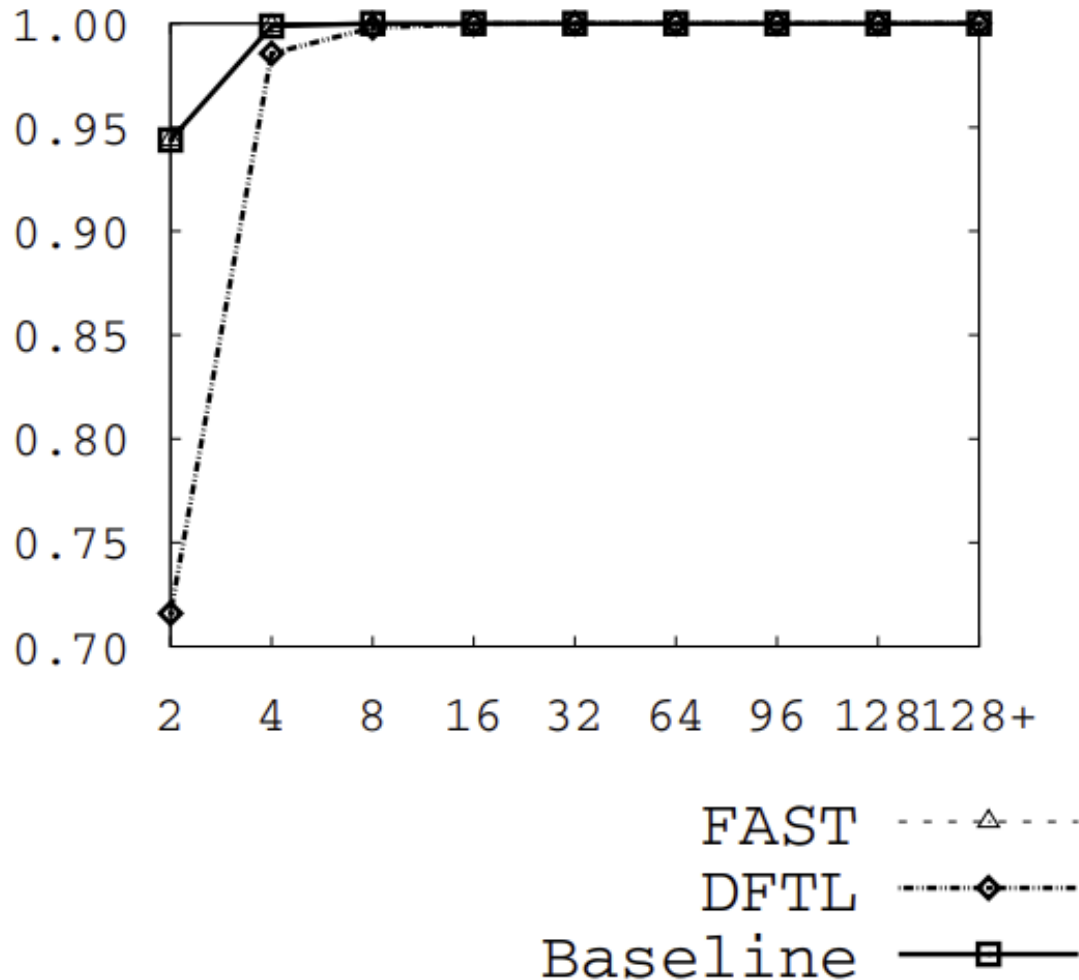
(TPC-H: Read 95% + 평균 요청 간격 155ms)

전체 95% Read 요청을 빠르게 처리해도
나머지 요청에서 긴 지연 발생 (꼬리 지연)

→ 누적 처리량 증가할수록 성능이 DFTL보다 낮아짐

Experiment Results

[Experiment 2: 성능 분석 (Web-Search)]



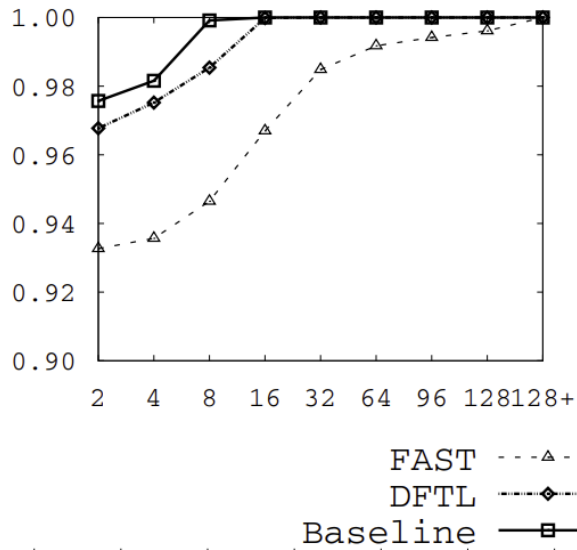
(Web-Search: Read 99% + 평균 요청 간격 10ms)

FAST는 베이스라인과 유사한 성능

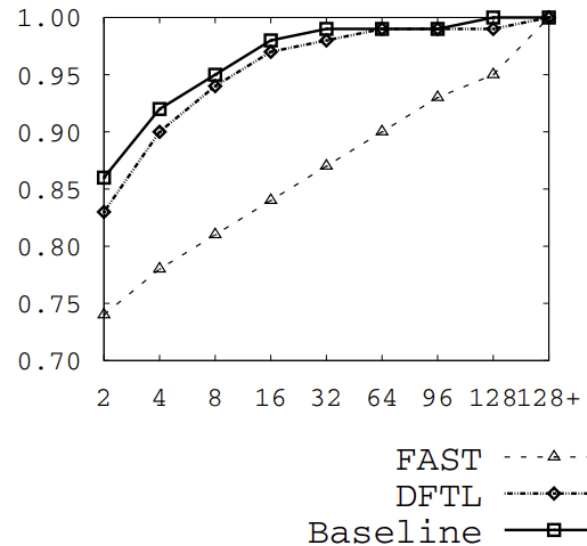
DFTL은 Full Merge 제거 효과보다
주소 변환 오버헤드의 영향이 더 크게 나타남

Experiment Results

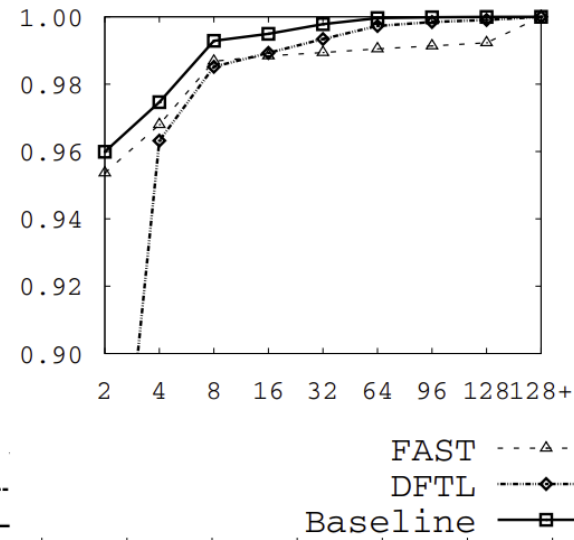
[Experiment 2: 성능 분석]



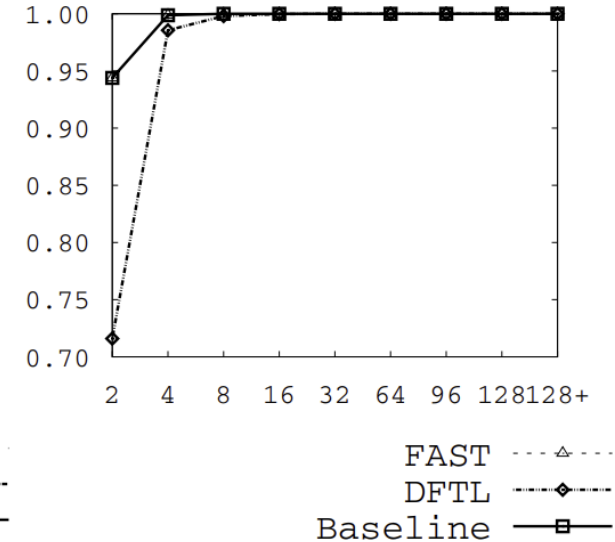
Financial (write)



Cello99 (write)



TPC-H (read)



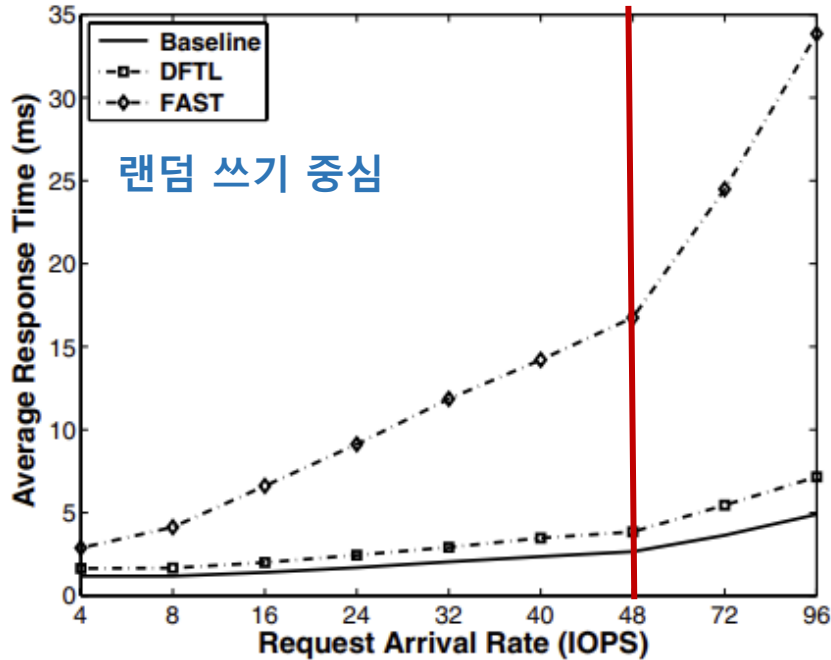
Web-Search (read)

DFTL은 Write 중심 워크로드에서 우수한 성능을 보이고, Read에서도 전반적으로 FAST와 큰 성능 차이를 보이지 않음

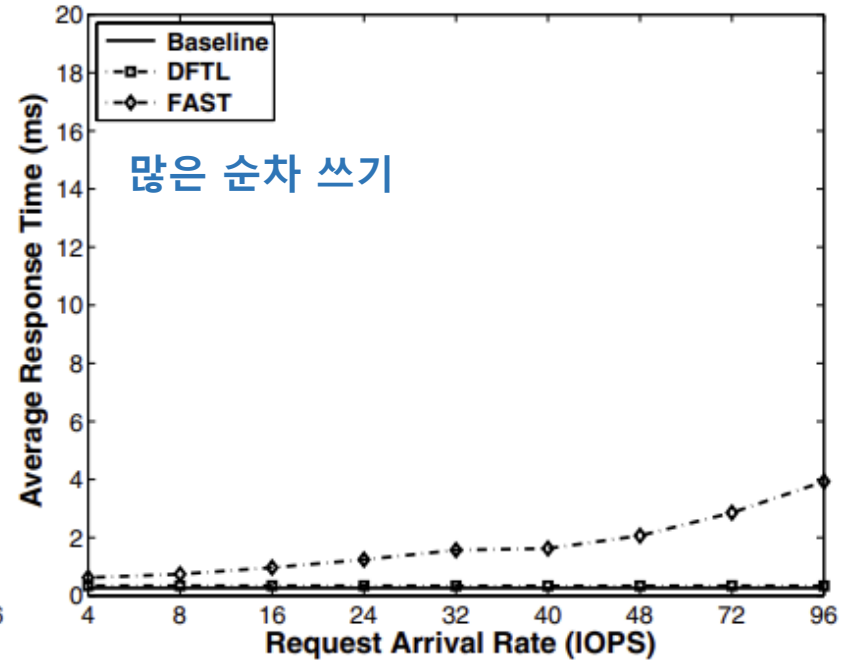
Experiment Results

[Experiment 3: 더 넓은 범위의 워크로드 특성 분석]

워크로드의 요청 도착률을 증가시키면
플래시 장치는 정상 동작 영역 → 과부하 영역으로 전환



(a) Workload A



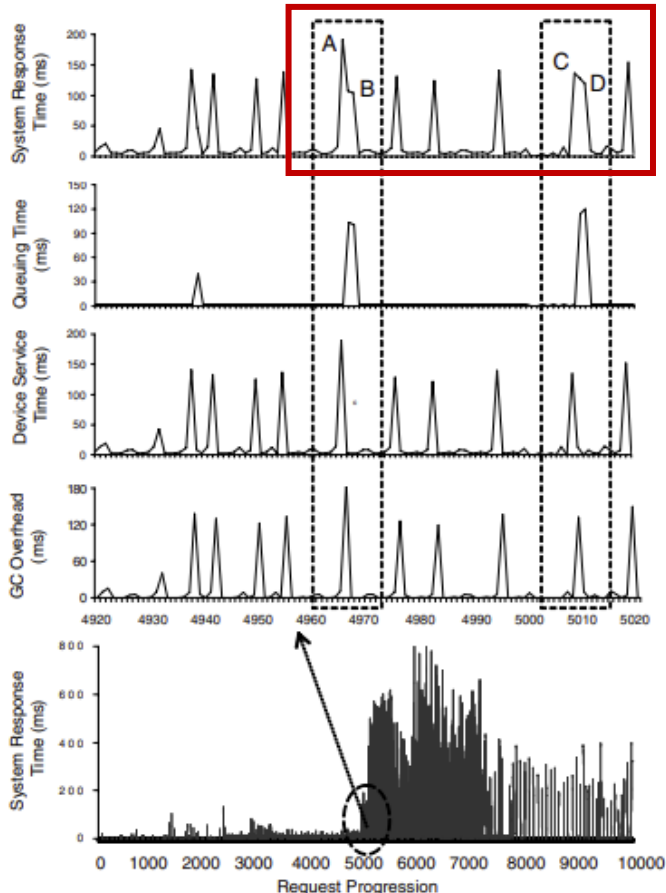
(b) Workload B

FAST는 부하를 지속적으로 처리하기 어려움

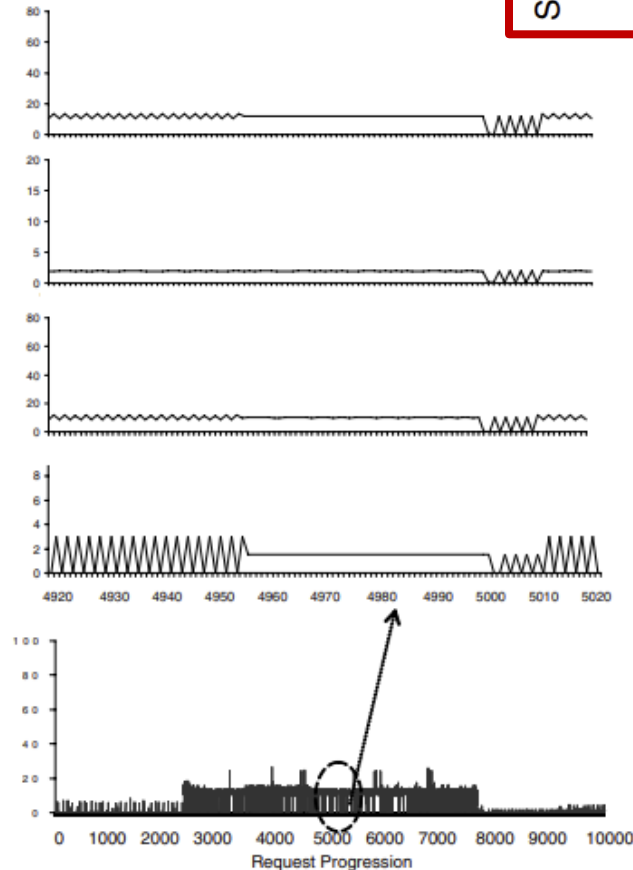
DFTL은 성능이 점진적으로 저하 → 엔터프라이즈 시스템에서 중요한 특성

Experiment Results

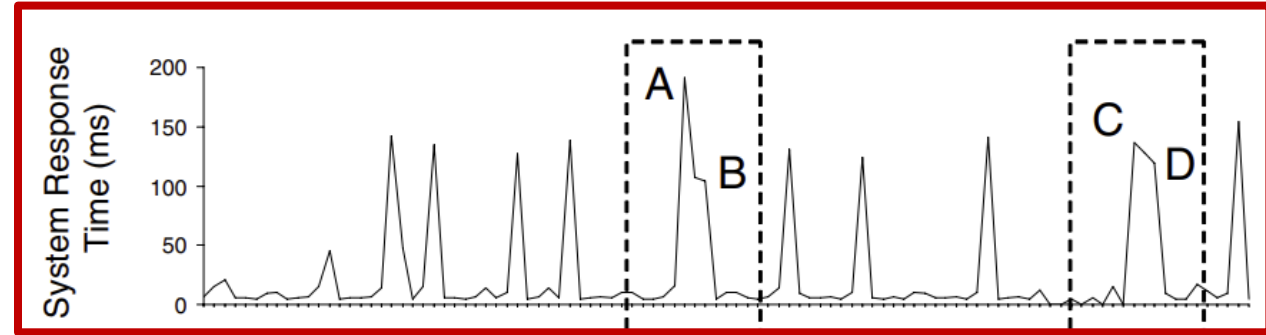
[Experiment 4: GC가 응답시간에 미치는 영향 분석]



(a) FAST



(b) DFTL

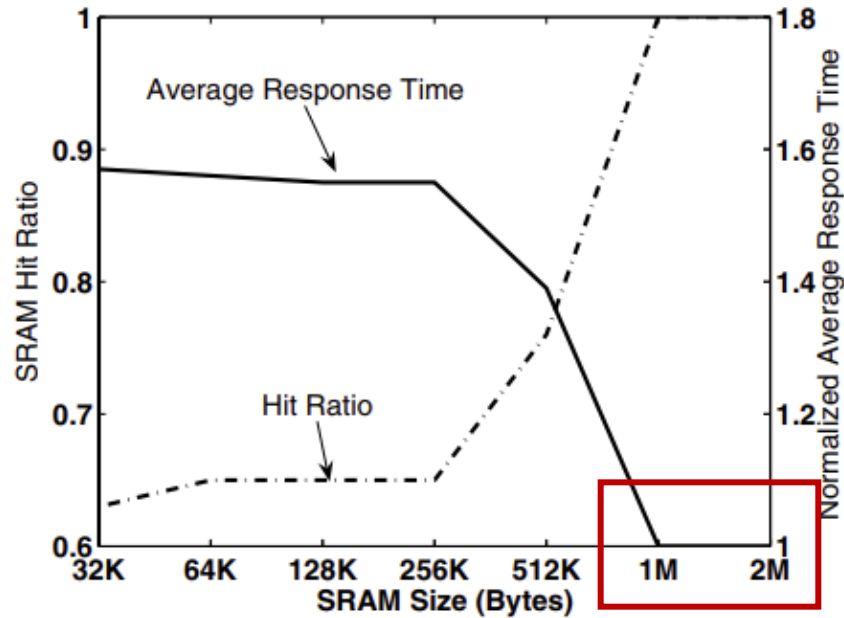


FAST: A에서 Full Merge 인한 복사/삭제 작업
 → 장치 서비스 시간 증가
 → I/O 드라이버 대기열에 있는 B 요청의 큐잉 시간 증가
 → 전체 시스템 응답 시간 증가

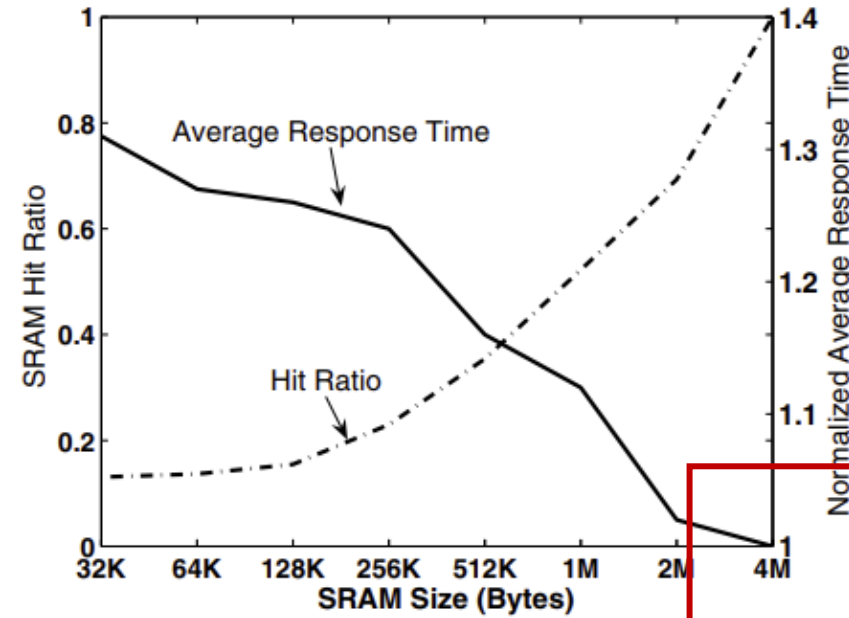
DFTL: Full Merge를 수행하지 않으므로 요청에 대한 향상된 성능을 지속적으로 제공

Experiment Results

[Experiment 5: SRAM 크기의 영향 분석]



(a) Financial Trace



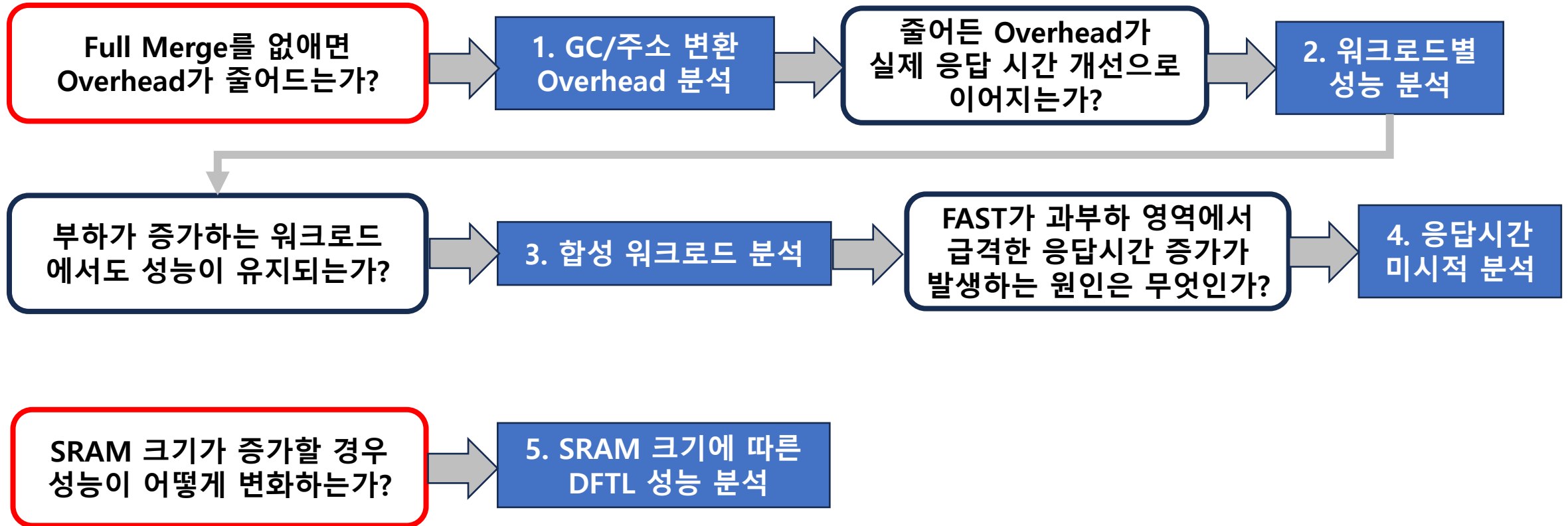
(b) TPC-H Benchmark

SRAM 크기가 증가 → CMT 적중률 향상, Overhead 감소 → 성능 향상

SRAM 크기가 워크로드의 작업 집합 크기를 넘어서면,
주소 변환을 위한 추가 SRAM은 더 이상 성능 향상 측면에서 이점을 제공하지 않음

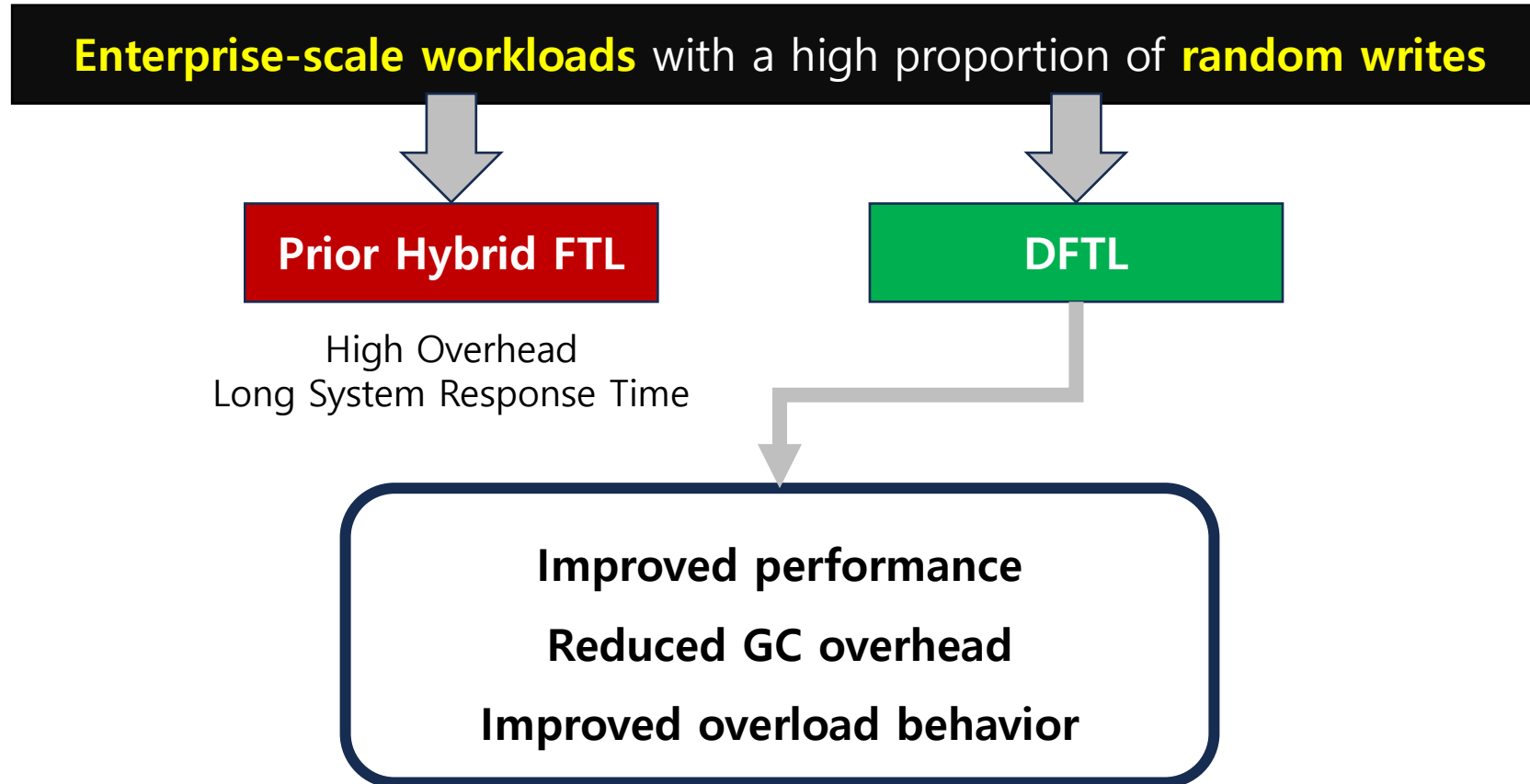
Experiment Results

[Experiment Relationships]



Full Merge 제거 + Overhead 감소 → DFTL은 다양한 워크로드와 과부하 환경에서 안정적인 성능을 제공

Conclusion



Thank you